



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



**НГТУ
НЭТИ**

**Факультет прикладной
математики
и информатики**

Кафедра прикладной математики
Практическое задание № 1
по дисциплине «Цифровые модели и оценивание параметров»

ЛИНЕЙНЫЕ ОБРАТНЫЕ ЗАДАЧИ

ПМ-34 КУДЮКОВ МАКСИМ
Вариант 2 ПМ-34 РУБЦОВ АРТЁМ

Преподаватель ВАГИН ДЕНИС ВЛАДИМИРОВИЧ

Новосибирск, 2026

Содержание

1 Задание.....	3
2 Постановка задачи	3
3 Входные данные	3
4 Теоретическая часть	3
5 Тестирование.....	4
5.1 Тестирование при трёх неизвестных I , без параметра регуляризации	4
6 Программа	4

1 Задание

Реализовать решение обратной задачи с выбранным вариантом.

2 Постановка задачи

Однородное полупространство. Приёмники 1–3. Источники 1–3.

Определить значения сил токов I_1 , I_2 , I_3 в источниках.

3 Входные данные

Положение источников	Положение приёмников
A1(0,–500,0), B1(100,–500,0); A2(0,0,0), B2(100,0,0); A3(0,500,0), B3(100,500,0).	M1(200,0,0), N1(300,0,0); M2(500,0,0), N2(600,0,0); M3(1000,0,0), N3(1100,0,0).

4 Теоретическая часть

$$\Phi(I) = \sum_i^{N^r} (w_i(V_i(I) - \bar{V}_i))^2 \frac{\partial(\delta V_i(I^n))}{\partial I_q} \frac{\partial(\delta V_i(I^n))}{\partial I_s}$$

$$V_{AB}^{MN} = \frac{I}{2\pi\sigma} \left(\left(\frac{1}{r_B^M} - \frac{1}{r_A^M} \right) - \left(\frac{1}{r_B^N} - \frac{1}{r_A^N} \right) \right).$$

Где r_B^M – расстояние между источником B и приёмником M
(у нас в задаче 3 приёмника и источника поэтому M и B будут иметь индексы)

$$r_{B_{1,2,3}}^{M_{1,2,3}}$$

Задача решается через метод Гаусса-Ньютона, поэтому нужно собрать систему и решить СЛАУ

$\Delta I = b$, система без регуляризации Тихонова, гдн

$$A_{qs} = \sum_i^{N^r} (w_i)^2 \frac{\partial(\delta V_i(I^n))}{\partial I_q} \frac{\partial(\delta V_i(I^n))}{\partial I_s}$$

$$b_q = \sum_i^{N^r} (w_i)^2 \delta V_i(I^n) \frac{\partial(\delta V_i(I^n))}{\partial I_q}$$

$$\frac{\partial(\delta V_i(I^n))}{\partial I_s} = \frac{V_i(I_s^n + \Delta I_s) - V_i(I_s^n - \Delta I_s)}{2\Delta I_s}$$

Система с регуляризацией Тихонова:

$$(\mathbf{A} + \alpha \mathbf{I}) \Delta \mathbf{p} = \mathbf{b} - \alpha(\mathbf{p}^n - \bar{\mathbf{p}})$$

α – коэффициент регуляризации, $\bar{\mathbf{p}}$ – неизвестно, можно брать равное $\mathbf{p}^n$

5 Тестирование

5.1 Тестирование при трёх неизвестных I, без параметра регуляризации

Начальное приближение $\bar{I}_0 = (2,2,2)$

$\sigma_0 = 0.01$; $I = (4.3, 10.1, 1.1)$

Итерация	I1	I2	I3	Total_Residual	Phi1	Phi2	Phi3
0	2	2	2	1.59869	0.286101	0.643172	0.669421
1	4.30	10.10	1.10	2.8165e-32	2.8165e-32	0.00	0.00

Решение находится за 1 итерацию, без регуляризации

Начальное приближение $\bar{I}_0 = (10,10,10)$

$\sigma_0 = 0.01$; $I = (1, 4, 8)$

Итерация	I1	I2	I3	Total_Residual	Phi1	Phi2	Phi3
0	10	10	10	83.3125	81	2.25	0.0625
1	1.00	4.00	8.00	0.00	0.00	0.00	0.00

Решение находится за 1 итерацию, без регуляризации

6 Вывод

Задача была решена без регуляризации и за одну итерацию из-за точных входных данных и хорошей обусловленности матрицы, в задачах, в задачах где входных данных меньше, появляется необходимость в регуляризации.

7 Программа

```
//main.cpp

#include "ReverseProblem.h"

int main()
{
    std::vector<double> trueI = { 1, 4 , 8 };
    ReverseProblem problem(3, trueI);

    try
    {
        problem.Solve();
    }
    catch (const std::runtime_error& e)
    {
        std::cout << "Caught a runtime error: " << e.what() << std::endl;
    }
    for (int i = 0; i < problem.n; i++)
    {
        std::cout << problem.I[i] << std::endl;
    }
}
```

```

    return 0;
}

#pragma once

#include <vector>
#include <iostream>
#include <cmath>
#include <numbers>
//ReverseProblem.h

class ReverseProblem
{
public:
    ReverseProblem(int _n, std::vector<double> _trueI)
    {
        n = _n;
        A.resize(n, std::vector<double>(n, 0));
        b.resize(n);
        dVdI.resize(n, std::vector<double>(n, 0));
        I.resize(n, 10.0);
        trueI = _trueI;
        trueV.resize(n);
        V.resize(n);
        w2.resize(n);
        residuals.resize(n);
        receiverMPos = { xyz(200,0,0), xyz(500,0,0), xyz(1000,0,0) };
        receiverNPos = { xyz(300,0,0), xyz(600,0,0), xyz(1100,0,0) };
        sourceAPos = { xyz(0,-500,0), xyz(0,0,0), xyz(0,500,0) };
        sourceBPos = { xyz(100,-500,0), xyz(100,0,0), xyz(100,500,0) };
    }

    struct xyz {

        xyz() : x(0), y(0), z(0) {}

        xyz(double _x, double _y, double _z) : x(_x), y(_y), z(_z) {}

        double x = 0;
        double y = 0;
        double z = 0;
    };

    int n = 0;
    std::vector<std::vector<double>> A;
    std::vector<double> b;
    std::vector<double> I;
    std::vector<double> trueI;
    std::vector<double> trueV;
    std::vector<double> V;
    std::vector<std::vector<double>> dVdI;
    std::vector<xyz> receiverMPos;
    std::vector<xyz> receiverNPos;
    std::vector<xyz> sourceAPos;
    std::vector<xyz> sourceBPos;
    std::vector<double> w2;
    std::vector<double> residuals;
    double alpha = 0;
    double sigma = 0.01;
    double eps = 1e-10;
    int maxIter = 10;

```

```

double Distance(xyz p1, xyz p2);
void ComputeTrueV();
void ComputeCurrentV();
void ComputeJacobian();
void WeightsInit();
void BuildMatrix();
void BuildRightPart();
void Regularisation();
double CalcResidual();
void CalcIndividualResiduals();
void Solve();

};

//ReverseProblem.cpp

#include "ReverseProblem.h"
#include "Gauss.h"

double ReverseProblem::Distance( xyz p1, xyz p2 )
{
    return sqrt( ( p2.x - p1.x ) * ( p2.x - p1.x ) +
        ( p2.y - p1.y ) * ( p2.y - p1.y ) +
        ( p2.z - p1.z ) * ( p2.z - p1.z ) );
}

void ReverseProblem::ComputeTrueV( )
{
    double piSigma = 0.5 * 1 / ( std::numbers::pi * sigma );
    for ( int i = 0; i < n; i++ )
    {
        trueV[i] = piSigma * trueI[i] * ( 1 / Distance( sourceBPos[i], receiverMPos[i] )
        - 1 / Distance( sourceAPos[i], receiverMPos[i] ) -
        ( 1 / Distance( sourceBPos[i], receiverNPos[i] ) - 1 / Distance(
        sourceAPos[i], receiverNPos[i] ) ) );
    }
}

void ReverseProblem::ComputeCurrentV( )
{
    double piSigma = 0.5 * 1 / ( std::numbers::pi * sigma );
    for ( int i = 0; i < n; i++ )
    {
        V[i] = piSigma * I[i] * ( 1 / Distance( sourceBPos[i], receiverMPos[i] ) - 1 /
        Distance( sourceAPos[i], receiverMPos[i] ) -
        ( 1 / Distance( sourceBPos[i], receiverNPos[i] ) - 1 / Distance(
        sourceAPos[i], receiverNPos[i] ) ) );
    }
}

void ReverseProblem::ComputeJacobian( )
{
    double piSigma = 0.5 * 1 / ( std::numbers::pi * sigma );
    for ( int i = 0; i < n; i++ )
        for ( int j = 0; j < n; j++ )
        {
            if ( i == j )
                dVdI[i][j] = piSigma * ( 1 / Distance( sourceBPos[j], receiverMPos[i] )
                - 1 / Distance( sourceAPos[j], receiverMPos[i] ) -
                ( 1 / Distance( sourceBPos[j], receiverNPos[i] ) - 1 / Distance(
                sourceAPos[j], receiverNPos[i] ) ) );
        }
}

```

```

        else
            dVdI[i][j] = 0;
    }
}

void ReverseProblem::WeightsInit( )
{
    for ( int i = 0; i < n; i++ )
        w2[i] = 1 / ( trueV[i] * trueV[i] );
}

void ReverseProblem::BuildMatrix( )
{
    for ( int q = 0; q < n; q++ )
        for ( int s = 0; s < n; s++ )
            for ( int i = 0; i < n; i++ )
                A[q][s] += w2[i] * dVdI[i][q] * dVdI[i][s];
}

void ReverseProblem::BuildRightPart( )
{
    for ( int q = 0; q < n; q++ )
        for ( int i = 0; i < n; i++ )
            b[q] += w2[i] * ( trueV[i] - V[i] ) * dVdI[i][q];
}

void ReverseProblem::Regularisation( )
{
    for ( int i = 0; i < n; i++ )
        A[i][i] += alpha;
}

double ReverseProblem::CalcResidual( )
{
    double res = 0;
    for ( int i = 0; i < n; i++ )
    {
        double diff = V[i] - trueV[i];
        res += w2[i] * diff * diff;
    }
    return res;
}

void ReverseProblem::CalcIndividualResiduals( )
{
    for ( int i = 0; i < n; i++ )
    {
        double diff = V[i] - trueV[i];
        residuals[i] = w2[i] * diff * diff;
    }
}

void ReverseProblem::Solve( )
{
    double residual = 1;
    Gauss gauss;
    ComputeTrueV( );
    WeightsInit( );
    ComputeJacobian( );

    std::cout << "Iter\tI1\tI2\tI3\tTotal_Residual\tPhi1\tPhi2\tPhi3" << std::endl;
}

```

```

    ComputeCurrentV( );
    residual = CalcResidual( );
    CalcIndividualResiduals( );
    std::cout << 0 << "\t" << I[0] << "\t" << I[1] << "\t" << I[2] << "\t" <<
residual << "\t";
    for ( int i = 0; i < n; i++ )
    {
        std::cout << residuals[i];
        if ( i < n - 1 ) std::cout << "\t";
    }
    std::cout << std::endl;

    for ( int iter = 0; iter < maxIter && residual > eps; iter++ )
    {
        for ( int i = 0; i < n; i++ )
            for ( int j = 0; j < n; j++ )
                A[i][j] = 0;

        for ( int i = 0; i < n; i++ )
            b[i] = 0;

        ComputeCurrentV( );
        BuildMatrix( );
        BuildRightPart( );
        Regularisation( );

        std::vector<double> deltaI = gauss.solve( A, b );

        for ( int i = 0; i < n; i++ )
        {
            I[i] += deltaI[i];
        }

        ComputeCurrentV( );
        residual = CalcResidual( );
        CalcIndividualResiduals( );
        std::cout << iter + 1 << "\t" << I[0] << "\t" << I[1] << "\t" << I[2] << "\t"
<< residual << "\t";
        for ( int i = 0; i < n; i++ )
        {
            std::cout << residuals[i];
            if ( i < n - 1 ) std::cout << "\t";
        }
        std::cout << std::endl;
    }
}

//Gauss.h

#include <iostream>
#include <vector>
#include <cmath>
#include <stdexcept>

using namespace std;

class Gauss {
public:
    vector<double> solve(vector<vector<double>> A, vector<double> b);
    void printMatrix(const vector<vector<double>>& A, const vector<double>& b);
};

```

```

//Gauss.cpp

#include "Gauss.h"

vector<double> Gauss::solve(vector<vector<double>> A, vector<double> b) {
    int n = A.size();

    if (n == 0 || A[0].size() != n || b.size() != n) {
        throw invalid_argument("Неверные размеры матрицы или вектора");
    }

    for (int k = 0; k < n; ++k) {
        int maxRow = k;
        double maxVal = fabs(A[k][k]);

        for (int i = k + 1; i < n; ++i) {
            if (fabs(A[i][k]) > maxVal) {
                maxVal = fabs(A[i][k]);
                maxRow = i;
            }
        }

        if (maxVal < 1e-10) {
            throw runtime_error("Division by zero");
        }

        if (maxRow != k) {
            swap(A[k], A[maxRow]);
            swap(b[k], b[maxRow]);
        }

        double pivot = A[k][k];
        for (int j = k; j < n; ++j) {
            A[k][j] /= pivot;
        }
        b[k] /= pivot;

        for (int i = k + 1; i < n; ++i) {
            double factor = A[i][k];
            for (int j = k; j < n; ++j) {
                A[i][j] -= factor * A[k][j];
            }
            b[i] -= factor * b[k];
        }
    }

    vector<double> x(n, 0.0);
    for (int i = n - 1; i >= 0; --i) {
        x[i] = b[i];
        for (int j = i + 1; j < n; ++j) {
            x[i] -= A[i][j] * x[j];
        }
    }

    return x;
}

void Gauss::printMatrix(const vector<vector<double>>& A, const vector<double>& b) {
    int n = A.size();
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cout << A[i][j] << "\t";
        }
    }
}

```

```
    }  
    cout << "|" << b[i] << endl;  
}  
cout << endl;  
}
```